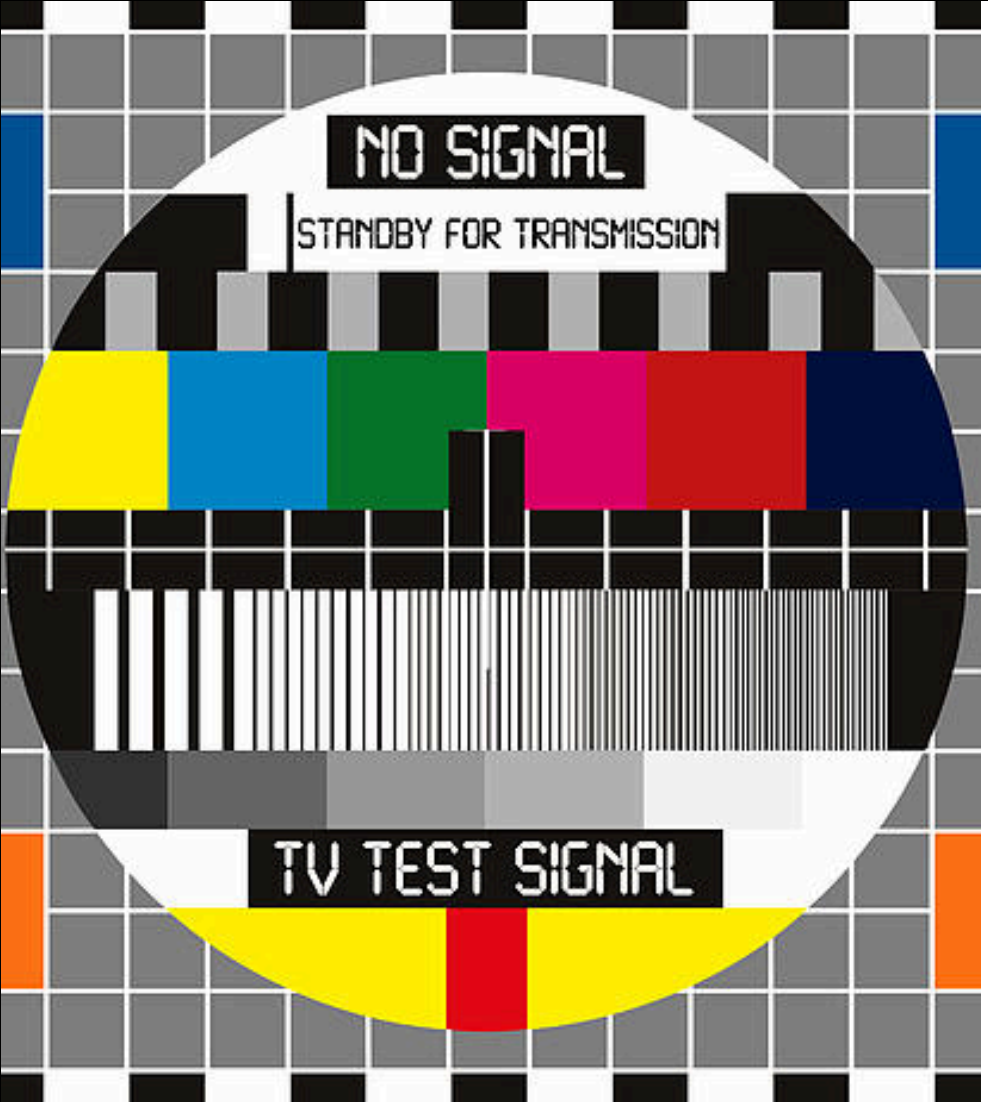


Streaming 101

From Pixels to Packets in JavaScript

timjs meetup, 2026

UP NEXT

I spy with my little eye

What is a video, and why our brains insist it's moving.

Thaumatrope (1825)

Thaumatropio cane e uccelli, 1825. Wikimedia Commons.

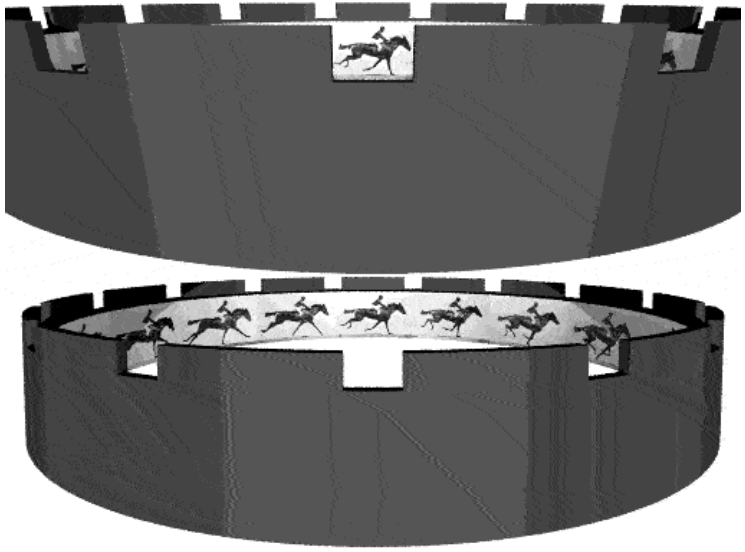
- **John Ayrton Paris**, a London surgeon-turned-toymaker.
- A small cardboard disc with one image on each side, spun on twisted strings.
- Each face flashes 10 to 20 times per second. The retina holds the previous image just long enough that you see them as **one combined picture**.
- First commercial proof of **persistence of vision**.

Phenakistoscope (1832)

Phenakistoscope disc, c. 1833. Wikimedia Commons.

- **Joseph Plateau** in Belgium, and **Simon Stampfer** in Austria, independently.
- A spinning disc with sequential drawings around the rim, viewed through slits in front of a mirror.
- The slits act as a shutter, masking each frame so only one is visible at a time. The first device to show **fluid motion**.
- Plateau is sometimes called the first animator. He spent his last 40 years blind, possibly from staring at the sun in his early experiments.

Zoetrope (1834)



Muybridge horse zoetrope simulation. Wikimedia Commons.

- **William George Horner**, English mathematician. Originally called the *daedalum*; renamed *zoetrope* (Greek "wheel of life") in 1867.
- A rotating drum with slits cut in the side; sequential images line the inside wall.
- The big upgrade over the phenakistoscope: **multiple people watch at the same time**.
- The **flipbook** (1868, kineograph) was the same trick on paper pages: flick your thumb, get motion.

Praxinoscope (1877)

Reynaud praxinoscope (CnAM 16554). Wikimedia Commons.

- **Émile Reynaud**, French inventor.
- Replaced the zoetrope's slits with a ring of mirrors. Reflections stitch the sequence together with **no flicker** and a brighter image.
- In 1888 his **Théâtre Optique** projected hand-painted strips for an audience: the first public animated projection, **eight years before** the Lumière brothers' first film show in 1895.

The Phi Phenomenon (1912)

Demo: youtube.com/watch?v=-zbzt7Cb2e4

- **Max Wertheimer**, founder of Gestalt psychology, 1912.
- Two stationary lights flashing in alternating positions are perceived as **one moving light**.
- Persistence of vision says "the last image lingers". Phi says **the brain invents the motion between distinct images**.
- Modern cinema relies on phi, not persistence. Each frame is a separate sharp picture; the visual cortex fills in the slide between them.

3D Holographic Fan: POV in 2026

POV LED fan demo. youtube.com/watch?v=fXqMqn2h9r8

- A ring of LEDs on **rotating fan blades**, spinning at 400 to 1000 RPM.
- The microcontroller fires each LED at the **exact angle** where that point should display a pixel of the image.
- Your eye smears the dots together into a **floating image** in mid-air.
- Same physics as the **phenakistoscope** (1832), with 200 years of LED engineering and a microcontroller.
- Modern uses: storefront ads, trade-show signage, Marvel-movie cameos.

You’re Blind Several Times a Minute

Your eyes don’t pan smoothly. They jump in **saccades**, 3 to 5 times every second.

- During each saccade (~30 to 80 ms) the brain **suppresses vision entirely** to hide the motion blur. *(Erdmann & Dodge, 1898; Volkmann, 1962.)*
- Add it up: roughly **40 minutes of every waking day, you are functionally blind**, and you don’t notice.
- The **stopped-clock illusion** is direct proof. Glance quickly at a wall clock and the second hand seems to pause an unusually long beat. Your brain back-fills the saccade with the *first* image it sees on landing. *(Yarrow et al., Nature, 2001.)*

> Your visual system drops frames constantly and lies about it.

Blinks Are P-Frames for Your Eyes

You blink 15 to 20 times per minute, ~100 ms each. Roughly **10% of your waking life is spent with your eyes shut.**

- The world doesn't go dark. The brain holds the last image in **iconic memory** for ~250 to 500 ms. *(George Sperling, Harvard PhD thesis, 1960.)*
- Whatever didn't change since the previous "frame" is reused. Whatever did is patched in.

Codec trick	Brain trick
I-frame: full picture, decodable alone	A fresh look on saccade landing
P-frame: store only what changed	Reuse iconic memory, patch the delta
Buffer: smooth out network jitter	Smooth out blinks and saccades

> Compression engineers reinvented, in software, the same shortcuts evolution shipped in our visual cortex.

You Live ~100 ms in the Past

Vision has a pipeline. Each stage adds latency.

- **13 to 20 ms**: photon hits the retina, triggers a chemical signal in a photoreceptor.
- **40 to 60 ms**: signal travels the optic nerve to the visual cortex.
- **100 to 150 ms**: cortex assembles edges, motion, depth into something you "see."

Streaming latencies, for comparison

- **Your brain**: ~100 ms.
- **WebRTC video call**: ~200 ms.
- **Low-Latency HLS**: ~1,000 ms.
- **Standard HLS / TV broadcast**: 6 to 30 seconds.

> "Real-time" is an illusion. Even your own eyes lag ~100 ms behind reality.

An error occurred on this slide. Check the terminal for more information.

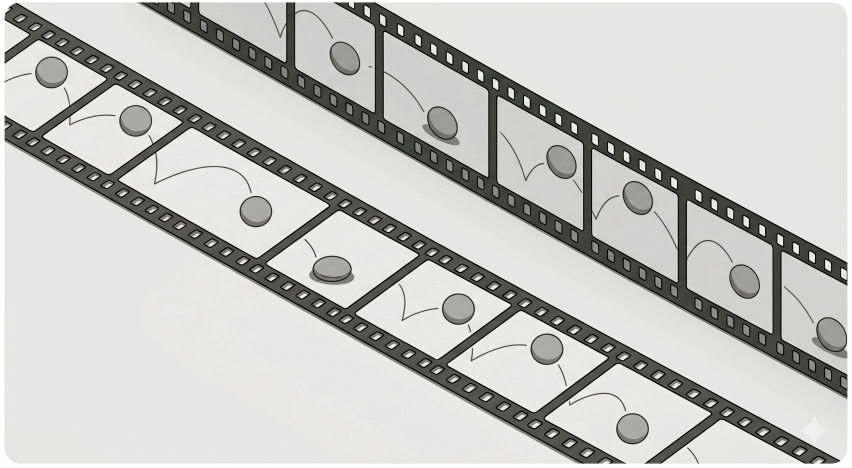
Your Brain Is Already Predicting

If the brain has a 100 ms lag, how do you catch a ball?

- The visual cortex **extrapolates**. It guesses where moving objects will be ~100 ms ahead of where they currently are, to compensate for its own delay.
- The **flash-lag effect** *(Nijhawan, Nature, 1994)*: a moving ball and a flash that *physically* coincide are perceived with the ball **ahead** of the flash. The brain has already moved on.
- This is exactly the same trick a P-frame uses, but in the *time* dimension instead of *space*.

> You're not seeing reality. You're seeing your brain's best guess of where reality is *about to be*.

How Big Is a Video?



At the lowest level: a sequence of **frames**, each frame a grid of **pixels**, each pixel a set of **bytes**.

Pixel → Frame → Video

Level	What it is	Size
Byte	8 bits (0 or 1), 0–255	1 byte
Pixel	3 bytes (R, G, B)	3 bytes
Frame	1920 × 1080 pixels	**~6 MB**
1 second	30 frames	**~180 MB/s**
1 minute	60 seconds	**~10.8 GB**

> Nobody streams raw video. This is why compression exists.

YouTube Quality Tiers: What Do They Mean?

The number is the **vertical pixel count**. More pixels = sharper image, but exponentially more data.

Label	Resolution	Pixels/Frame	×1080p
360p	640 × 360	230,400 ≈ 230K	0.11×
480p (SD)	854 × 480	409,920 ≈ 410K	0.20×
720p (HD)	1280 × 720	921,600 ≈ 922K	0.44×
1080p (Full HD)	1920 × 1080	2,073,600 ≈ 2.07M	1×
1440p (2K)	2560 × 1440	3,686,400 ≈ 3.69M	1.78×
2160p (4K)	3840 × 2160	8,294,400 ≈ 8.29M	4×

> "4K" = **4× the pixels** of 1080p, not 4× the width.

Why "p" and Why Vertical?

- The **"p"** stands for **progressive scan** (every line drawn each frame, vs **"i"** = interlaced, odd/even lines alternating)
- Early TV standards were defined by **scan lines** (vertical resolution): 480i (NTSC), 576i (PAL)
- When HD arrived, the same convention stuck: **720p**, **1080i**, **1080p**
- The **horizontal** pixels just follow from the **aspect ratio** (16:9): given 1080 vertical → 1920 horizontal

So why is 2160p called "4K"?

- Cinema (DCI) standard is 4096 × 2160, and the **"4K"** refers to ~4000 **horizontal** pixels
- Consumer "4K" (UHD) is 3840 × 2160. The name was borrowed from cinema marketing.
- Confusingly, **4K switches to horizontal** naming while everything else uses vertical

Frame Rate: 30 fps vs 60 fps

The **frame rate** multiplies everything. More frames per second = smoother motion, but double the data.

Resolution	30 fps (raw)	60 fps (raw)	Difference
720p	82,944,000 ≈ 79 MB/s	165,888,000 ≈ 158 MB/s	2×
1080p	186,624,000 ≈ 178 MB/s	373,248,000 ≈ 356 MB/s	2×
4K	746,496,000 ≈ 712 MB/s	1,492,992,000 ≈ 1.4 GB/s	2×

- **30 fps**: standard for most video (films are 24 fps)
- **60 fps**: gaming, sports, fast motion (YouTube shows "1080p60" badge)
- Higher fps helps with **motion clarity** but doesn’t improve still-image sharpness

UP NEXT

Codecs Are Magic

How a 1.4 Gbps stream becomes 5 Mbps without you noticing.

The Math Behind Raw Video

A single 1080p frame at 30 fps:

$$R_{\text{raw}} = 1920 \times 1080 \times 3 \times 30 \approx 178\ \text{MB/s} \approx 1.42\ \text{Gbps}$$

What compression buys us

Codec	Compression	1080p30 Bitrate	1 hour
Raw	1:1	1.42 Gbps	625 GB
H.264	~50:1	~28 Mbps	~12.5 GB
H.265	~100:1	~14 Mbps	~6.25 GB
AV1	~130:1	~11 Mbps	~4.8 GB

Vocab: encode, decode, codec

- **code**: from Latin *codex*, "a system of rules". Same root as *encrypt*, *cipher*, *codify*.
- **encode** = put **into** code. (`en-` = "into")
- **decode** = take **out of** code. (`de-` = "away from")
- **codec** = a 1970s telephony portmanteau: **CO**(der) + **DEC**(oder).
  - Anything that does both: turn raw pixels into compressed bits, *and* the reverse.

> Encoder lives at the **producer** (camera, server). Decoder lives at the **consumer** (player, monitor). Same algorithm, run backwards.

Containers vs Codecs

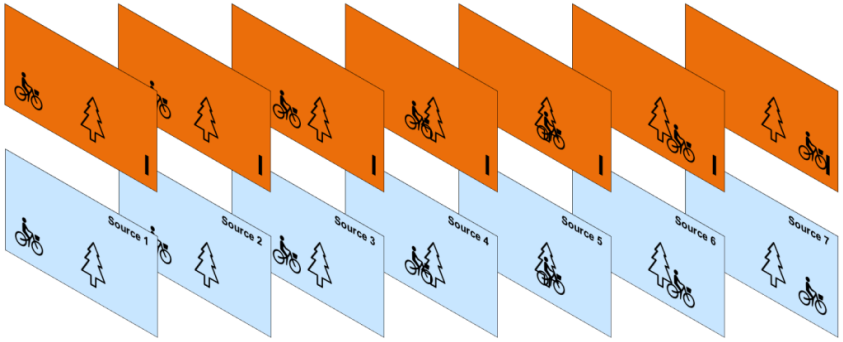
- **Container** (industry term: *wrapper format*) = MP4, WebM, MKV, TS. Holds multiple **streams**: video, audio, subtitles, plus sync metadata.
- **Codec** = the algorithm each stream was compressed with: H.264, H.265, VP9, AV1, Opus, AAC.

Mnemonic: MP4 is to H.264 what ZIP is to deflate

- A `.zip` is a **container**. Inside, each file can use a different compression algorithm (deflate, lzma, store).
- A `.mp4` is the same idea: one outer file, several inner streams, each compressed with its own codec.
- The extension tells you the **container**. Only inspecting the bytes (`ffprobe`) tells you the **codecs** inside.

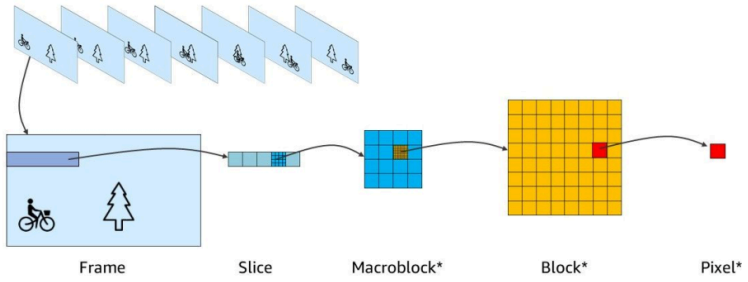
> When a video won't play, ask: *which* container *and which* codec? Browsers reject combinations they don't understand even when both halves are individually fine.

Motion JPEG: Each Frame On Its Own



- The simplest video codec: JPEG-compress every frame.
- Kills **spatial** redundancy (within each frame).
- Ignores that frames 1 and 2 are nearly identical. A bicyclist riding past a tree reprints the tree seven times.

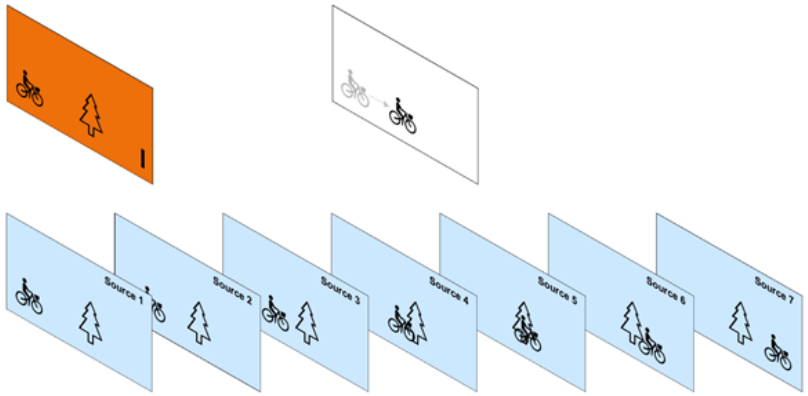
MPEG: Frame, Slice, Macroblock, Block, Pixel



*Not always square. (This graphic is simplified.)
May be rectangular, depending on the codec used.

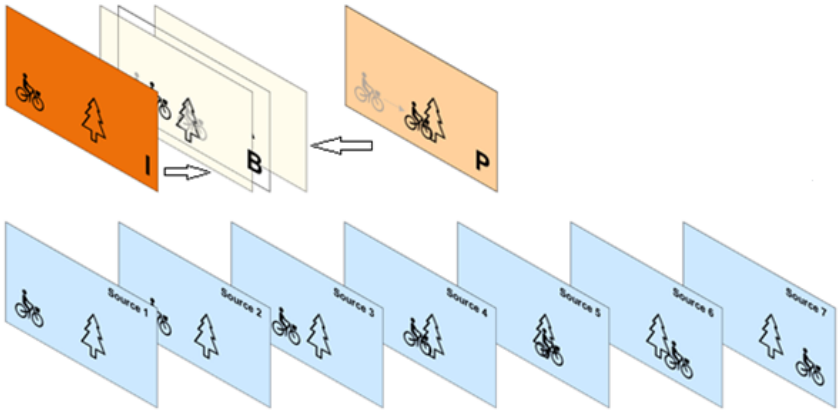
- Where Motion JPEG stops at the frame, the **MPEG family** (H.264, H.265, AV1) exploits what's similar *between* frames.
- To do that, it first breaks each frame into a hierarchy:
  - **Slice**: an independently decodable strip of the frame.
  - **Macroblock**: the basic unit of motion estimation (typically 16×16 pixels).
  - **Block**: the unit of DCT / transform coding inside a macroblock.
  - **Pixel**: the leaf.
- Everything that follows (motion vectors, I/P/B frame types, quantization) operates on **macroblocks**, not whole frames.

Motion Estimation: Predict, Don’t Store



- Frames are divided into **macroblocks** (small pixel tiles).
- For each block, the encoder searches the previous frame for the best match.
- Stored: a **motion vector** ("moved +3, +1") plus a tiny **error term**. Orders of magnitude smaller than raw pixels.
- Heart of every MPEG-family codec (H.264, H.265, AV1).

I, P, and B Frames



- **I** (intra): complete picture, JPEG-like. Decodable alone. Biggest.
- **P** (predicted): references one earlier frame. "Same as before, but this block moved."
- **B** (bidirectional): references **past and future**. Smallest of the three.

> An H.264 stream is mostly B-frames, with occasional P-frames, anchored by rare I-frames.

But Wait: Cameras Can’t See the Future

The trick: lookahead

The encoder buffers a few frames before emitting any output. It pretends "the future" already happened by holding back a tiny bit.

- More lookahead → better compression, more latency.
- Less lookahead → lower latency, fatter files.

Live often skips B-frames entirely

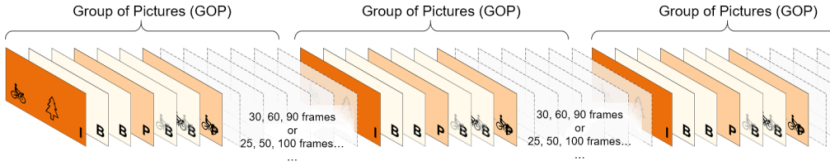
For real-time scenarios, encoders run with `-tune zerolatency` (or equivalent), which **disables B-frames**. You trade compression efficiency for ~0 frames of encode delay.

Where does the actual encoding run?

- Not C++ on the camera. **Fixed-function silicon** on the SoC: Apple VideoToolbox, Qualcomm Venus, Samsung MFC, Intel Quick Sync.
- The C/C++ part is the OS driver that feeds frames in and pulls compressed bytes out.

An error occurred on this slide. Check the terminal for more information.

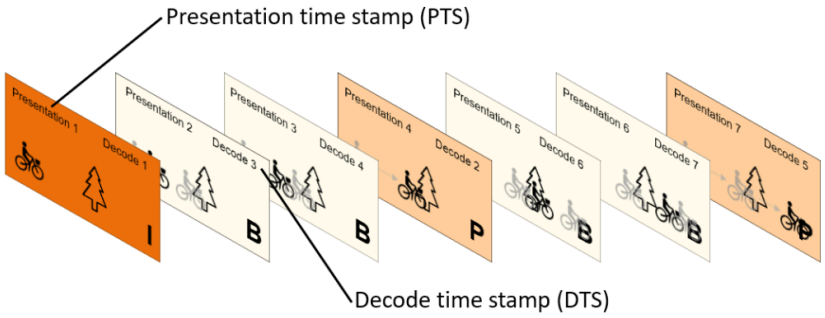
GOP: Group of Pictures



- A **GOP** is the repeating pattern from one I-frame to the next. Typical sizes: **30, 60, or 90 frames** (1 to 3s at 30 fps).
- **Bigger GOP**: better compression, slower seek, longer recovery after a dropped packet.
- **Smaller GOP**: faster seek, fatter files. Live uses short, Netflix uses long.

> HLS segments must start on an I-frame: **GOP size bounds minimum segment duration**.

Decode Order ≠ Display Order



- A **B-frame** can only be decoded after its past *and* future references exist. Arrival order ≠ display order.
- Every frame carries two timestamps: **PTS** (present) and **DTS** (decode).
- Player decodes in DTS order, reorders by PTS before rendering. Seeking rewinds to the nearest I-frame and decodes forward.

When Does Compression Actually Happen?

- **Is the uploaded file already compressed?** Yes. Your phone's camera encodes H.264 in real time. The raw 1.4 Gbps sensor stream is squeezed to ~10 Mbps **before** it touches disk.
- **Is it stored compressed?** Yes, at every stage. The server keeps the uploaded original *and* the re-encoded variants, all compressed.
- **Does compression happen during transmission?** No. HTTP just moves the same compressed bytes. Nothing is re-encoded in flight.
- **Are the lower-quality variants "the compression"?** Yes. Each variant (480p, 720p, 1080p) is a **separate re-encode** at a different target resolution and bitrate.
- So compression runs **twice** in our pipeline: once in the camera at capture, once on the server at transcode. Playback is strictly *decompression*.

An error occurred on this slide. Check the terminal for more information.

Transcoding: One File, Many Variants

The algorithm, per variant

1. **Decode** source bytes to raw pixel frames.
2. **Rescale** to the target resolution (1920×1080 → 854×480).
3. **Re-encode** with the target codec + bitrate (H.264 @ 800 kbps for 480p).

Repeat for 720p and 1080p. Three passes = three independent compressed files.

What our ffmpeg command does

```
ffmpeg -i source.mp4 \
  -vf scale=854:480 \
  -c:v libx264 -b:v 800k \
  -f hls 480p/stream.m3u8
```

Why not just lower the bitrate?

- Keeping 1080p at 800 kbps looks **blocky**. The codec has too many pixels and too few bits.
- Rescaling first gives the codec fewer pixels to care about, so every bit it spends shows up as detail.

Bandwidth vs Quality Tradeoff

Quality follows a **logarithmic** curve with bitrate:

- Doubling bitrate does **not** double quality
- Going from 1 → 2 Mbps is far more noticeable than 10 → 11 Mbps
- Below a threshold: quality drops catastrophically (the "potato zone" 🥔)

Connection	Bandwidth	Max Quality
3G Mobile	~2 Mbps	480p
4G Mobile	~20 Mbps	1080p
Home Wi-Fi	~50 Mbps	4K
Covered phone 📱🤚	<1 Mbps	Buffering

FFmpeg: The Swiss Army Knife of Video

- **Created in 2000 by Fabrice Bellard** (also wrote QEMU, TinyCC, and calculated π to a record number of digits).
- **FF** = *Fast Forward*, the tape-deck button. **mpeg** = the codec family it first targeted.
- A set of C **libraries** (`libavcodec` , `libavformat` , `libavfilter` , `libswscale`) plus three CLIs: `ffmpeg` (transcode), `ffprobe` (inspect), `ffplay` (play).
- **Hundreds of codecs, dozens of containers, one unified API.** Before FFmpeg you needed a different tool per format.

Where you find it

- VLC, MPV, OBS, Kodi: all built on `libav*` .
- Twitch, Netflix, YouTube transcoding pipelines: FFmpeg under the hood.
- Chrome and Firefox use FFmpeg-derived code for `<video>` decoding.

> Our server uses it **twice**: re-encoding uploads into HLS variants (`videos.service.ts`), and converting the live WebM stream into H.264 segments (`streams.service.ts`).

Fabrice Bellard

- Born 1972, Grenoble. École Polytechnique, then Télécom Paris.
- 1989 *(age 17)*: writes **LZEXE**, an executable compressor for DOS.
- 1997 *(age 25)*: derives the **Bellard formula** for π, ~47% faster than BBP.
- 2000 *(age 28)*: starts **FFmpeg**.
- 2003 *(age 31)*: starts **QEMU**. Modern hardware virtualization (KVM) is built on top of it.
- 2011 *(age 39)*: **JSLinux**, a Linux PC running inside a browser tab.
- 2014 *(age 42)*: **BPG**, a JPEG replacement built on H.265 intra-frame coding.
- 2019 *(age 47)*: **QuickJS**, a tiny embeddable JavaScript engine.

Records and recognition

- 2009 *(age 37)* **π record**: ~**2.7×10^{12}** digits on a single ~$3,000 desktop. Beat **$2.58 \times 10^{12}$** (Daisuke Takahashi, Univ. of Tsukuba T2K, Aug 2009).
- For scale, today's record sits at **2.02×10^{14}** digits (StorageReview / Jordan Ranous, 2024), about **75×** Bellard's, on a server with ~1 PB of storage.
- 2011 **O'Reilly Open Source Award** for QEMU. Three **IOCCC** *(International Obfuscated C Code Contest)* wins.

Day job

- Co-founded **Amarisoft** in 2012 *(age 40)*, in Levallois-Perret (Paris). Software 4G/5G base stations on commodity hardware.
- Still CTO. Almost no public profile: virtually no interviews, no social media.

Image: Computer History Museum

The Checkerboard Demo

- A **static** checkerboard compresses almost perfectly. Temporal compression removes everything.
- A **rotating** checkerboard defeats temporal compression; every frame is unique.
- At low bitrates, sharp edges show **blocking artifacts**: the squares smear into gray zones.

> This is exactly what happens when your player switches from high to low quality.

Making the `_still` Files with FFmpeg

```
# 1. Static checkerboard PNG (geq filter draws each square via pixel coords)
ffmpeg -f lavfi -i "color=white:s=1080x1080" \
  -vf "geq=lum='if(eq(mod(floor(X/135)+floor(Y/135),2),0),255,0)'" \
  -frames:v 1 checkerboard.png

# 2. Loop the still PNG into a 10s video (no rotation, no motion at all)
ffmpeg -loop 1 -i checkerboard.png -t 10 \
  -c:v libx264 -pix_fmt yuv420p -r 30 \
  checkerboard_still.mp4

# 3. Re-encode the still clip at three bitrate tiers
ffmpeg -i checkerboard_still.mp4 -c:v libx264 -b:v 5M   high_still.mp4
ffmpeg -i checkerboard_still.mp4 -c:v libx264 -b:v 200k low_still.mp4
ffmpeg -i checkerboard_still.mp4 -c:v libx264 -b:v 50k  potato_still.mp4
```

> Start simple: nothing moves. Every frame is identical to the first, so temporal compression flattens all four files to **under 60 KB**, regardless of the target bitrate.

Making the `_rotating` Files with FFmpeg

```
# 1. Rotate the PNG (360 degrees over 10s, defeats temporal compression)
ffmpeg -loop 1 -i checkerboard.png -t 10 \
  -vf "rotate=2*PI*t/10:fillcolor=white:ow=1080:oh=1080" \
  -c:v libx264 -pix_fmt yuv420p checkerboard_rotating.mp4

# 2. Re-encode the rotating clip at the same three bitrate tiers
ffmpeg -i checkerboard_rotating.mp4 -c:v libx264 -b:v 5M   high_rotating.mp4
ffmpeg -i checkerboard_rotating.mp4 -c:v libx264 -b:v 200k low_rotating.mp4
ffmpeg -i checkerboard_rotating.mp4 -c:v libx264 -b:v 50k  potato_rotating.mp4

# 3. Stack two clips horizontally for side-by-side viewing
ffmpeg -i high_rotating.mp4 -i potato_rotating.mp4 \
  -filter_complex "[0:v][1:v]hstack=inputs=2" comparison.mp4
```

> Now make it hard: every frame is unique. The encoder has nothing to reuse, so the **100× bitrate spread** (5 Mbps vs 50 kbps) translates straight into the visible artifacts on the next slide.

Same Codec, Wildly Different Sizes

Every clip below is **H.264 / libx264**, profile **High**, level **3.2**, `yuv420p`, **30 fps**, **1080×1080**, **10 s**. Only the encoder flags change.

File	Source	Encoder flag	Bitrate	Size
`checkerboard.png`	`lavfi` + `geq`	PNG (lossless)	n/a	26 KB
`checkerboard_still.mp4`	looped PNG	default (CRF 23)	16 kbps	24 KB
`checkerboard_rotating.mp4`	rotated PNG	default (CRF 23)	1.56 Mbps	**1.95 MB**
`high_still.mp4`	still re-encode	`-b:v 5M`	23 kbps	33 KB
`high_rotating.mp4`	rotating re-encode	`-b:v 5M`	4.47 Mbps	**5.60 MB**
`low_still.mp4`	still re-encode	`-b:v 200k`	42 kbps	57 KB
`low_rotating.mp4`	rotating re-encode	`-b:v 200k`	356 kbps	169 KB
`potato_still.mp4`	still re-encode	`-b:v 50k`	34 kbps	46 KB
`potato_rotating.mp4`	rotating re-encode	`-b:v 50k`	305 kbps	385 KB

What is CRF 23? CRF = *Constant Rate Factor*, libx264's quality-based mode. The encoder targets a **perceived quality level** (0 = lossless, 51 = unwatchable, 23 = the default sane choice; 18 is "visually lossless"). File size depends on **how complex the content is**: a static frame at CRF 23 is tiny, a rotating checkerboard at the same CRF blows up to 1.56 Mbps.

Why is `high_rotating.mp4` larger than the source? The source used CRF 23 → ~1.56 Mbps. `high_rotating.mp4` was forced to a 5 Mbps target, so the encoder spends 3× more bits. Re-encoding upward inflates the file but **cannot recover** information the source already discarded.

UP NEXT

Introducing the video tag

One HTML element, every file format the browser feels like supporting.

The `<video>` Tag: A Brief History

- **Before 2010**: Video on the web meant **Flash Player**, QuickTime, or RealPlayer (browser plugins)
- **2007**: Opera proposes a native `<video>` element for HTML5
- **2010**: Steve Jobs publishes "Thoughts on Flash"; Apple refuses Flash on iOS
- **2011**: Major browsers ship `<video>` support (Chrome, Firefox, Safari, IE9)
- **2012**: YouTube offers HTML5 player as opt-in beta
- **2015**: YouTube defaults to HTML5, Netflix drops Silverlight for HTML5
- **2017**: Flash officially deprecated by Adobe
- **2020**: Flash Player end-of-life. The `<video>` tag won.

> One HTML tag replaced an entire ecosystem of plugins.

From 3 Lines of HTML to Full Control

The simplest possible video player:

```
<video src="movie.mp4" controls width="640"></video>
```

But under the hood, this one tag triggers:

- **Network**: HTTP range requests, chunked downloads, caching
- **Demuxing**: Separating video/audio/subtitle tracks from the container
- **Decoding**: Decompressing H.264/VP9/AV1 frames (CPU or GPU)
- **Rendering**: Compositing decoded frames onto the screen via the GPU
- **Audio sync**: Keeping audio and video in lockstep (A/V sync)
- **DRM**: Encrypted Media Extensions (EME) for protected content

What Happens When You Press Play

- **JavaScript** calls `video.play()`. That's the last "easy" part.
- The browser's **C++ media pipeline** takes over (Chromium: `media/`, Firefox: `dom/media/`)
- A **demuxer** (C++) reads the container format and extracts compressed frames
- Frames are sent to a **decoder**: either software (C/C++ via FFmpeg/libvpx) or hardware (GPU)
- Decoded frames land in a **compositor** that paints them to screen via the GPU

The language boundary

- **JavaScript**: play/pause, volume, currentTime, events, MediaSource API
- **C/C++**: demuxing, decoding, buffering, A/V sync, DRM decryption
- **GPU (shader code)**: color space conversion (YUV → RGB), scaling, compositing

An error occurred on this slide. Check the terminal for more information.

Hardware vs Software Decoding

- **Software decoding**: CPU runs C/C++ codec libraries (FFmpeg, libvpx, dav1d)
- **Hardware decoding**: Dedicated silicon on the GPU handles it (NVDEC, VideoToolbox, VAAPI)

	Software (CPU)	Hardware (GPU)
Speed	Slower (general-purpose cores)	Much faster (purpose-built circuits)
Power	High CPU usage, battery drain	Minimal (dedicated low-power block)
4K60	Struggles on older CPUs	Effortless on modern GPUs
Codec support	Everything (just compile it)	Limited to what chip supports
Fallback	Always available	Falls back to software if unsupported

> Check `chrome://media-internals` to see which decoder your browser chose.

Does a Better GPU Help?

For decoding, not really (past a baseline)

- Decoding uses a **fixed-function block** (NVDEC, not CUDA cores); a $200 GPU decodes 4K just as well as a $1500 one
- What matters: does the GPU support the **codec**? (e.g., AV1 hardware decode needs RTX 30xx+ or Intel Arc)
- More VRAM or CUDA cores do **not** help video playback

For display quality, the monitor matters more

- **Color vibrancy**: determined by the **panel** (IPS, OLED, HDR support), not the GPU
- **HDR tone mapping**: GPU + OS work together (Windows HDR, macOS EDR)
- **Color accuracy**: ICC profiles managed by the **OS**, calibrated per-display
- **Scaling/sharpness**: GPU handles upscaling (FSR, DLSS), but the browser mostly doesn't use these

Who Owns What: Browser vs OS vs Hardware

Responsibility	Who handles it
Container parsing (MP4, WebM)	**Browser**, C++ media stack
Codec decoding (H.264, AV1)	**Browser → GPU**, hardware if available, else CPU
Color space conversion (YUV→RGB)	**GPU**, shader or fixed-function
Color management (ICC profiles)	**OS**, color profiles per display
HDR tone mapping	**OS + GPU driver** (Windows HDR / macOS EDR)
Frame presentation (vsync, tearing)	**OS compositor** (DWM on Win, Quartz on Mac)
DRM decryption (Widevine, FairPlay)	**Browser CDM**, sandboxed C++ module
Audio output & sync	**Browser → OS** via Web Audio / platform APIs

> The `<video>` tag is JavaScript's thin interface to a **deep C++/GPU/OS stack**.

The `<video>` API Surface

What JavaScript actually controls:

- **Playback**: `play()`, `pause()`, `playbackRate`, `currentTime`
- **Source**: `src`, `<source>` elements, `MediaSource` API (for hls.js)
- **Tracks**: `audioTracks`, `textTracks`, `videoTracks`
- **Events**: `timeupdate`, `waiting`, `canplay`, `error`, `ended`
- **Capture**: `captureStream()` grabs frames as a `MediaStream`
- **Canvas bridge**: draw video frames to `<canvas>` for pixel manipulation

What JavaScript cannot control:

- Which decoder (hardware vs software) is used
- Color management or ICC profile selection
- GPU memory allocation or frame scheduling
- A/V sync algorithm internals

UP NEXT

Chunked, Cached, Adaptive

HLS chops video into bites the network and the CDN can survive.

Meet HLS: HTTP Live Streaming

- **HLS** = HTTP Live Streaming, Apple's protocol for delivering video over plain HTTP/HTTPS.
- Shipped with **iPhone OS 3** in **June 2009** so the iPhone could stream over flaky 3G.
- Standardized as **RFC 8216** (IETF Informational, August 2017) eight years after launch.
- The trick: chop the video into small files, list them in a playlist, let any **HTTP server or CDN** serve them. No streaming server, no special protocol.

Two file types do all the work

- `.m3u8` — the playlist. A UTF-8 text file (`.m3u` from the Winamp era + `8` for UTF-8) that lists either quality variants or segment URLs.
- `.ts` — MPEG-2 Transport Stream segment. A few seconds of audio + video. Originally designed for satellite/cable broadcast in the 1990s; HLS reused it because every device already decoded it.
- **fMP4** — added to HLS in **2016** so it could share segments with MPEG-DASH (the rival ISO/IEC standard from 2012). Same playlist format, different container.

> One playlist file, a pile of small video chunks, ordinary HTTP. That is the entire idea. The next 20 slides are details.

An error occurred on this slide. Check the terminal for more information.

What's an `.m3u8` File?

- **M3U** = "MP3 URL", a playlist format from the Winamp era (1990s).
- **M3U8** = M3U encoded in **UTF-8** (the "8" is the encoding, not a version number).
- Apple adopted it for HLS. It's just a **text file** listing chunk URLs.

HLS uses two flavors

- **Master playlist**: lists the quality *variants* (one entry per resolution/bitrate). Loaded once at startup.
- **Media playlist**: lists the actual `.ts` segments for **one** variant. Refetched periodically for live.

Master Playlist: Quality Variants

```
#EXTM3U
#EXT-X-STREAM-INF:BANDWIDTH=800000,RESOLUTION=640x360
360p/playlist.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=2800000,RESOLUTION=1280x720
720p/playlist.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=5000000,RESOLUTION=1920x1080
1080p/playlist.m3u8
```

- `#EXTM3U` (line 1): magic header. Every M3U-family file starts with this.
- `#EXT-X-STREAM-INF` : declares a variant. `BANDWIDTH` is the target in bps, `RESOLUTION` is pixels.
- The line **after** each `STREAM-INF` is the URL of that variant's media playlist.
- ABR uses these `BANDWIDTH` numbers to decide which variant to request next.

Reading the BANDWIDTH Numbers

`BANDWIDTH` is **bits per second**, the same unit ISPs quote (and 8× smaller than the bytes per second your file manager shows).

```
BANDWIDTH=800000   =  800,000 bps   =  800 kbps  =  0.8 Mbps   (360p)
BANDWIDTH=2800000  =  2,800,000 bps =  2.8 Mbps             (720p)
BANDWIDTH=5000000  =  5,000,000 bps =  5.0 Mbps             (1080p)
```

How that compares to real links

Link	Typical sustained speed	Variants that fit
3G phone	~2 Mbps	360p only
4G LTE	20 to 50 Mbps	1080p with room
5G mid-band	100 to 400 Mbps	20 to 80 simultaneous 1080p
Digi fibre 1 Gbps (RO)	~800 Mbps wired, ~300 over Wi-Fi	hundreds of 1080p

Pen and paper

- Digi sells "1 Gbps". In practice the line tops out around 800 Mbps over Ethernet (Wi-Fi cuts that further).
- One 1080p stream wants 5 Mbps. So **800 ÷ 5 ≈ 160 simultaneous 1080p streams** before the line saturates.
- 5G at 200 Mbps gives **200 ÷ 5 = 40** simultaneous 1080p streams. One viewer barely notices the load.
- ABR's real job isn't "find anything that fits". It's "find the highest variant that **survives one slow chunk** without draining the buffer".

Media Playlist: The Segments

```
#EXTM3U
#EXT-X-TARGETDURATION:6
#EXTINF:6.000,
segment-000.ts
#EXTINF:6.000,
segment-001.ts
#EXTINF:4.120,
segment-002.ts
#EXT-X-ENDLIST
```

- `#EXT-X-TARGETDURATION:6` : max segment length the player should expect.
- `#EXTINF:6.000,` + filename: this segment lasts 6.000 seconds.
- Next segment, also 6 seconds.
- The last one is **shorter** (4.120s) because the source video didn't divide evenly.
- `#EXT-X-ENDLIST` : VOD marker. "No more segments coming." Live playlists omit this.

What's a `.ts` Segment?

- **TS** = **MPEG-2 Transport Stream** (ISO/IEC 13818-1, 1995).
- Designed for **digital TV broadcast** where packets get lost over the air. Had to be self-syncing.
- Format: 188-byte packets, each tagged with a stream ID. A receiver can tune in mid-stream and recover within a few packets.
- HLS reuses TS because those same properties (chunked, joinable, error-tolerant) are exactly what HTTP delivery wants.

Why "segment"?

- One `.ts` file = one slice of the timeline (typically 2 to 6 seconds).
- Stitch many segments back-to-back and you reconstruct the whole video.
- VOD segments are immutable. Live segments are appended at the head and (optionally) deleted from the tail.

> Modern HLS also supports fMP4 segments (`.m4s`), but `.ts` is still the workhorse.

The HLS Browser Gap

HLS is an Apple-born standard. Support is fragmented across the desktop web.

Native Support (Safari/iOS)

- `<video src="stream.m3u8">` works out of the box.
- Browser handles everything (fetch, buffer, quality).
- **Cons:** Very limited control or telemetry.

No Native Support (Chrome/Firefox/Edge)

- Browser has no idea how to parse `.m3u8`.
- Loading it as a `src` results in an error.
- **Solution:** We need a JavaScript "driver" to teach the browser HLS.

hls.js: The JavaScript Driver

`hls.js` is a massive library because it re-implements the entire video stack in JS.

- **Fetch**: Uses standard `fetch` / `XHR` to download manifests and segments.
- **Transmuxing**: HLS often uses `.ts` (MPEG-TS). Browsers prefer `.mp4` (ISO BMFF). `hls.js` converts bytes on-the-fly in a Web Worker.
- **MSE (Media Source Extensions)**: The "Manual Pump" API that feeds raw bytes into the `<video>` element.
- **ABR Logic**: It monitors your bandwidth and decides when to switch qualities.

> Without `hls.js`, HLS streaming would fail for ~80% of desktop users.

Media Source Extensions (MSE)

The `<video>` tag was designed for **one file → one video**. But streaming needs to feed chunks dynamically.

The problem

- `<video src="movie.mp4">` loads one file, with no way to switch quality mid-stream
- HLS/DASH need to fetch **small chunks** and stitch them together on the fly
- The browser has no built-in HLS support (except Safari)

MSE: the solution (2013, W3C spec)

- JavaScript creates a `MediaSource` object and wires it to `<video>` via a blob URL
- Opens a `SourceBuffer`, a pipe where JS can **push raw media chunks**
- The browser's C++ decoder processes each chunk as if it were part of one continuous file
- This is **exactly** what hls.js does: it's an MSE client

MSE: How hls.js Wires It Up

```
// 1. Create a MediaSource and connect it to <video>
const ms = new MediaSource();
video.src = URL.createObjectURL(ms);

// 2. When ready, open a SourceBuffer for the codec
ms.addEventListener('sourceopen', () => {
  const sb = ms.addSourceBuffer('video/mp4; codecs="avc1.64001f"');

  // 3. Fetch an HLS chunk and push it in
  fetch('/hls/720p/segment-001.ts')
    .then(r => r.arrayBuffer())
    .then(data => sb.appendBuffer(data));
});
```

What this unlocks

- **Adaptive bitrate**: switch quality by pushing chunks from a different playlist
- **Live streaming**: keep appending new chunks as they arrive
- **Seeking**: jump to any point by fetching the right chunk and appending it
- **Gap handling**: detect buffering gaps and fetch missing segments

> Without MSE, libraries like hls.js, dash.js, and Shaka Player **could not exist**.

An error occurred on this slide. Check the terminal for more information.

Will Browsers Ever Add Native HLS?

> Short answer: no, and that's by design.

Browser	Native HLS?	Notes
Safari / iOS	Yes, since 2009	Apple owns the spec. Native LL-HLS too.
Chrome / Edge	No	No plans. Stable position for a decade.
Firefox	No	Same.

Why not?

- W3C / Chromium / Mozilla picked **MSE** over bundling protocols. "Give JS the primitives; let userland implement HLS, DASH, or whatever's next."
- HLS is an Apple-authored **informational** RFC (8216), not a multi-vendor W3C/ISO spec. Vendors don't want a single-vendor format baked in.
- DRM (FairPlay vs Widevine vs PlayReady) and codec licensing make a single native pipeline messy.

Where to read this for yourself

- **Mozilla** `standards-positions` (GitHub): search issues for "HLS".
- **Chromium issue tracker**: search "native HLS support" (long "won't fix, use MSE + hls.js" thread).
- **IETF RFC 8216**: the HLS spec itself. Note its *informational* status.
- **W3C MSE** spec: the official "this is the answer instead" document.

What's actually moving

- **WebCodecs** (Chromium shipped, others partial): direct access to hardware encode/decode. Even lower-level than MSE.
- **MoQ** (Media over QUIC): the IETF's next-gen streaming protocol on HTTP/3. If anything ships natively in non-Apple browsers, it'd be this, not HLS.
- **Managed MSE** (Safari 17+): battery-friendly hints. Augments the JS-library model, doesn't replace it.

> hls.js is here for the long haul.

What MSE Actually Enabled

The bet "ship MSE, not protocols" produced two streaming formats and a small ecosystem of JS players.

Two protocols ride on top of MSE

- **HLS** *(Apple, RFC 8216, 2009)*: `.m3u8` text playlists + `.ts` (or fMP4) segments. The de-facto default for live and OTT video on the open web.
- **MPEG-DASH** *(ISO/IEC 23009-1, 2012)*: vendor-neutral equivalent. XML manifest (`.mpd`), codec-agnostic. The "open standard" answer to HLS.

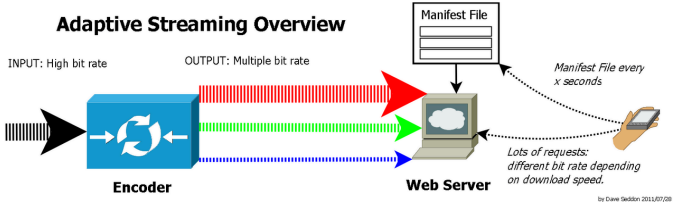
The JS players that decode them

Library	Plays	What it solves	In production at
hls.js	HLS	Polyfills HLS for Chrome / Firefox / Edge (Safari plays it natively)	**Twitch**, JW Player, DAZN
dash.js	DASH	Reference player from the DASH Industry Forum	**BBC iPlayer**, Akamai demos
Shaka Player	HLS + DASH + EME	One library for both protocols, plus DRM (Widevine / PlayReady / FairPlay) for paid content	**YouTube TV**, Google web products

> Netflix and YouTube run their own in-house players, but the *protocols* on the wire are still these two.

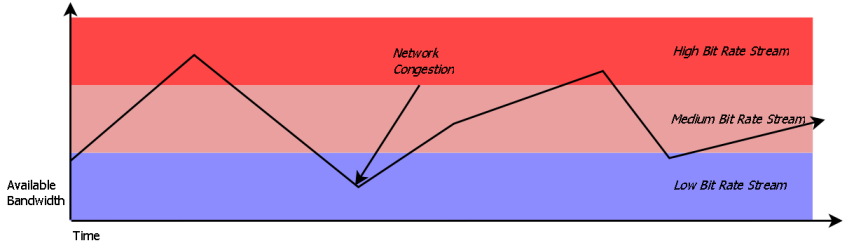
Adaptive Bitrate Streaming (ABR)

- The video isn't sent as one file. It's **chopped into small chunks** (2 to 6 seconds).
- The server provides a **manifest** (`.m3u8`) listing all quality variants and their chunks.
- The player measures **download speed** in real-time and picks the best quality for the next chunk.



Client jumps between variants as bandwidth changes. Wikimedia Commons, 2011.

ABR vs Network Bandwidth



Source content is encoded once per quality, producing parallel *bit rate ladders*. Wikimedia Commons, 2011.

- Each row is the **same content**, encoded at a different bitrate.
- The manifest publishes each row's bitrate via the `BANDWIDTH` field.
- The player matches its **measured download speed** against those numbers and picks the highest row that still fits.

The ABR Rule, in English

> Pick the highest-quality variant you can still download faster than it plays.

Every few seconds the player asks three questions:

1. **How fast did I just download the last chunk?** That's my estimated bandwidth.
2. **Which variants fit under that bandwidth?** Anything whose bitrate is lower.
3. **Which of those looks best?** Pick it; that's the next chunk.

The same thing in one line

$$Q^* = \arg\max_{q} \; Q(q) \quad \text{s.t.} \quad R(q) \le B_{\text{estimated}}$$

Symbol	Means
`q`	one of the variants in the manifest (480p, 720p, 1080p, …)
`R(q)`	bitrate that variant needs (the `BANDWIDTH` field in `.m3u8`)
`Q(q)`	how good that variant looks (grows with bitrate, logarithmic)
$B_{\text{estimated}}$	your measured download speed
`Q*`	the winner: best `Q(q)` whose `R(q)` still fits under `B`

ABR in Practice

Your phone is on 4G and the player measures **4 Mbps**. The manifest offers three variants:

Variant	Bitrate `R(q)`	Fits under 4 Mbps?	Picked
480p	0.8 Mbps	yes	no (looks worse)
720p	2.8 Mbps	yes	**← Q***
1080p	5.0 Mbps	no (would stall)	no

Real players add a safety margin

Using ~80% of estimated bandwidth leaves headroom so a single slow chunk doesn't empty the buffer and stall playback.

What happens at the edges

- **Pick too high** (1080p at 4 Mbps): chunk takes longer than 4s, buffer drains, spinner appears.
- **Pick too low** (480p at 4 Mbps): safe, but wastes 3.2 Mbps of headroom and looks worse than it needs to.
- **Bandwidth drops mid-stream** (elevator, weak Wi-Fi): next measurement is lower, ABR picks a lower variant for the next chunk. That's the YouTube quality downshift.

An error occurred on this slide. Check the terminal for more information.

UP NEXT

Live Is Always Late

When the next chunk doesn't exist yet, and how we cope.

Browser Caching: VOD vs Live

HLS chunks are just HTTP responses, so the browser (and CDNs) can cache them. But the caching strategy is **opposite** for VOD and live.

	VOD	Live
Chunks (.ts)	Immutable, cache forever	Immutable, cache but short-lived on disk
Manifest (.m3u8)	Static, cache aggressively	Changes every segment, **must not cache**
Cache-Control	`max-age=31536000`	`no-cache` or `max-age=1`
Seeking	Any chunk instantly (cached)	Only recent window (old chunks expire)
Replay	Free, served from cache	Impossible unless DVR window configured

> VOD = cache everything. Live = cache chunks, **never** cache the manifest.

An error occurred on this slide. Check the terminal for more information.

Live Streaming

- **Ingest**: Camera → WebRTC/MediaRecorder → WebSocket binary → Server
- **Transcode**: FFmpeg encodes to 1080p + 720p + 480p simultaneously
- **Deliver**: HLS chunks generated on-the-fly → viewers pull via `.m3u8` manifest

Two Clocks Tug at the Stream

Streaming is a fight between two clocks that wish they were identical.

- **Producer clock** *(at the encoder)*: ticks every time a frame is captured. Steady at, say, 30 Hz.
- **Consumer clock** *(at the browser GPU)*: ticks every time a frame is drawn. Also 30 Hz.
- The **network** between them is a *time stretcher*. Two packets sent 33 ms apart can arrive 200 ms apart, or 5 ms apart.

The conveyor-belt metaphor

- The encoder puts 30 boxes per second onto a belt.
- The player takes 30 boxes per second off the belt.
- If the belt's speed is perfectly constant, the buffer never fills or drains.
- The internet is **not a belt**. It is a series of unreliable couriers, each taking a slightly different amount of time.

> The buffer's job is to absorb the difference between the belt the encoder thinks it's on and the one the network actually provides.

Latency vs Jitter: The Real Killer

> Latency is **fixed delay**. Jitter is **variable delay**. Continuity dies on jitter, not latency.

	Latency	Jitter
What it is	How long any one packet takes	How much packet times *vary*
Example	Every packet: 200 ms	100, 900, 50, 300 ms ...
Effect on player	Just starts 200 ms later. Smooth.	"I needed a frame 30 ms ago. Where is it?"
Mitigation	None needed	Pre-buffer enough seconds to absorb the worst spike

- A perfectly slow link (high latency, zero jitter) plays smoothly. It just sits behind reality.
- A perfectly fast link with bursts (low latency, high jitter) **stutters** unless the buffer is big enough.
- "Average bandwidth" is a misleading number for live. Worst-case **arrival variance** matters more.

> A slow, steady stream beats a fast, bursty one. The buffer is a *shock absorber*, not a battery.

An error occurred on this slide. Check the terminal for more information.

Segment Size vs Latency

HLS requires **~3 chunks buffered** before playback starts.

Segment Size	Chunks Buffered	Latency
6 seconds	3	**~18s**
2 seconds	3	**~6s**
1 second	3	**~3s**

Shorter segments = lower latency, but more HTTP requests and less compression efficiency.

Why "Live" Is 7 to 30 Seconds Behind

"Live" TV is a polite fiction. By the time a goal hits your TV, the stadium has already cheered.

Stage	Cost	Why
Encoding	1 to 2 s	Lookahead for B-frames, motion estimation
Segmentation	~6 s	HLS player wants 3 segments before play. 2 s × 3 = 6 s.
CDN propagation	1 to 2 s	Origin to edge hops; cache fill on first request
Player buffer	2 to 10 s	Headroom for network jitter
Total	**10 to 30 s**	Standard HLS, end to end

The faster alternatives

- **Low-Latency HLS (LL-HLS)**: streams partial segments before the full one is finished. End-to-end ~1 s.
- **WebRTC**: skips HLS entirely, runs over UDP with no segmentation. ~200 ms. Different protocol, no CDN scale.

> HLS's bargain: **pay 7 to 30 s in latency, get infinite scale via plain HTTP CDNs**. For most live, that trade is fine.

An error occurred on this slide. Check the terminal for more information.

The Rolling Window

FFmpeg's "live" HLS mode is a **sliding window** of recent segments.

- `-hls_time 4` : each segment covers ~4 seconds of video
- `-hls_list_size 5` : manifest lists only the most recent 5 segments
- `-hls_flags delete_segments` : older `.ts` files are **erased from disk**

Net effect: at any moment, only the last **~20 seconds** of the stream exists. Anything older is gone; the server has no memory.

Playlist Types

`#EXT-X-PLAYLIST-TYPE` tells the player what kind of manifest this is.

	VOD	EVENT	LIVE *(no tag)*
Segments can be removed?	No	No	Yes
Playlist grows?	No (fixed)	Yes (append)	Sliding window
Player can scrub back?	Full timeline	Full timeline	Only current window
Ends with `#EXT-X-ENDLIST`?	Yes	When done	Never
FFmpeg flag	`-hls_playlist_type vod`	`-hls_playlist_type event`	default

`EVENT` is the sweet spot for live-with-DVR: append-only, scrubbable, and becomes a finished playlist when the stream ends.

DVR: Three Flags Flipped

DVR = *Digital Video Recorder*. Borrowed from late-90s set-top boxes like TiVo, where a hard drive let you pause, rewind and scrub a broadcast. In HLS, "DVR" means a live stream where the server keeps a long enough backlog of segments that the player can scrub back into the past instead of being stuck at the live edge.

```
// Before: sliding window, ~20s of history
'-hls_list_size', '5',
'-hls_flags', 'delete_segments+append_list',

// After: full DVR + auto-archive
'-hls_playlist_type', 'event',
```

- `-hls_list_size` → implicitly `0` (unbounded) under EVENT mode
- No more `delete_segments`; every `.ts` file stays on disk
- When stdin closes, FFmpeg writes `#EXT-X-ENDLIST` → playlist is now a complete VOD

Archive: Live → VOD, No Re-encode

When the camera stops, the recording is **already on disk** as HLS segments.

1. Close ffmpeg's stdin → it finalizes the last segment + writes `#EXT-X-ENDLIST`
2. `mv /data/hls/live/{id} → /data/hls/vod/{id}`
3. Register the directory as a new `Video` entity
4. Broadcast `transcode:complete` → catalog refreshes

No transcoding. No copying. Just a filesystem rename, and the exact same bytes are now a VOD.

An error occurred on this slide. Check the terminal for more information.

Storage Layout

```
/data/hls/
├── live/                          ← only while recording
│   └── {streamId}/
│       ├── master.m3u8
│       └── 720p/stream.m3u8 + segments
├── vod/                           ← permanent
│   ├── {uploadedVideoId}/         ← from Upload → transcoded
│   └── {archivedStreamId}/        ← from Live → renamed
│       ├── master.m3u8            ← ends with #EXT-X-ENDLIST
│       └── 720p/stream.m3u8 + segments
```

The server rebuilds `VideosService`'s in-memory catalog from `vod/` at startup, so archived streams survive restarts for free.

UP NEXT

One more thing...

Takeaways and your questions.

My Takeaways

Video is a magic trick
showing pictures fast enough that the brain invents the motion.

There is a lot of engineering
behind every video file, from the capture device to storage to playback.

Everyone uses FFmpeg
under the hood for video pipelines (Fabrice Bellard is the GOAT).

The network adds complexity
jitter, latency, ABR, all in service of keeping the picture moving.

"Live" is never live
neither in your biology nor in your browser.

Limited by design
browser standards have to please everybody, and the constraints aren’t always obvious.

Further Reading

HLS Specification (RFC 8216): the RFC behind HTTP Live Streaming

hls.js: the player library powering our viewer

FFmpeg Documentation: the transcoding engine reference

Web API: MediaRecorder: browser API for capturing camera streams

Adaptive Streaming (Wikipedia): overview of ABR techniques

Choose Your Next Adventure

WebRTC

real-time peer-to-peer streaming with sub-second latency

DASH

MPEG's alternative to HLS (Dynamic Adaptive Streaming over HTTP)

AV1

next-gen codec, better than H.265, royalty-free

WebTransport

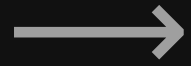
HTTP/3 based low-latency streaming protocol

Media Source Extensions

the browser API that makes hls.js possible

WebCodecs

low-level encode/decode directly in the browser



Live Demo

/presenter

SWITCH TO THE APPLICATION

Buffering Questions...

Open the floor.

github.com/rtodea/streaming-101